Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Bách khoa
Khoa Khoa học và Kỹ thuật Máy tính



Học sâu và ứng dụng trong thị giác máy tính (CO5085)

Bài tập lớn A2 - Mốc A2.1: Chủ đề, paper và dữ liệu

Phân đoạn ngữ nghĩa 38 lớp trên Oxford-IIIT Pet:
U-Net (CNN) hay SegFormer (Transformer)?

Giảng viên: Thầy Lê Thành Sách
Thầy Nguyễn Quốc Minh

Thành viên nhóm: Đàm Quang Phục - 2570483
Hồ Bảo An - 2570164

Mã nguồn:
`https://github.com/dqpinfo/Deep_Learning_Computer_Vision_CO5085/tree/main/Assignment_2`

Thành Phố Hồ Chí Minh, 11/2026

Mục lục

1 Chủ đề và bài toán **2**
1.1 Bài toán . . . 2
1.2 Vì sao 38 lớp mà không chỉ tách thú khỏi nền . . . 2
1.3 Câu hỏi nghiên cứu . . . 2

2 Paper phân tích: SegFormer **3**
2.1 Thông tin trích dẫn . . . 3
2.2 Lý do chọn . . . 3
2.3 Tóm tắt sơ bộ (sẽ phân tích đầy đủ ở A2.2) . . . 3

3 Tập dữ liệu: Oxford-IIIT Pet **4**
3.1 Nguồn, license và cách tải . . . 4
3.2 Cách chia split . . . 4
3.3 Khám phá dữ liệu (EDA) . . . 4

4 DataLoader và augmentation **8**

5 Metric đánh giá **9**

6 Hai phương pháp và kế hoạch pipeline **9**
6.1 Mô hình . . . 9
6.2 Huấn luyện . . . 10
6.3 Ma trận thí nghiệm A2.2 . . . 10
6.4 Phân tích sẽ báo cáo ở A2.2 . . . 10

7 Kiểm tra pipeline ở mốc A2.1 **10**

8 Kế hoạch đến mốc A2.2 **11**

9 Sử dụng công cụ AI **12**

Tóm tắt mốc A2.1 (hạn 02/11/2026). Nhóm chốt chủ đề **phân đoạn ngữ nghĩa** (semantic segmentation) trên tập **Oxford-IIIT Pet** [2], với bài toán **38 lớp**: nền và 37 giống chó mèo. Paper phân tích là **SegFormer** (Xie và cộng sự, NeurIPS 2021) [1]. Hai phương pháp so sánh là **U-Net với encoder ResNet-50** (CNN) và **SegFormer-B2** (Transformer), cùng được huấn luyện trước trên ImageNet-1k và có số tham số tương đương. Metric chính là **mIoU** trên 38 lớp; kèm mean Dice, pixel accuracy, IoU thú/nền và chi phí tính toán. Dữ liệu đã được tải và kiểm tra MD5, chia split cố định, EDA và DataLoader đã chạy thật; cả hai mô hình đã qua kiểm tra pipeline. Kết quả huấn luyện thuộc mốc A2.2 (16/11/2026). Mọi con số trong báo cáo lấy từ `config.py`, `output/eda/summary.json` và `output/protocol_table.json`.

1. Chủ đề và bài toán

1.1. Bài toán

Phân đoạn ngữ nghĩa là gán cho *mỗi điểm ảnh* một nhãn lớp. Khác với phân loại ảnh ở A1 (một nhãn cho cả ảnh), mô hình phải trả lời đồng thời "ở đâu" và "là gì". Nhóm định nghĩa bài toán trên Oxford-IIIT Pet như sau:

- **Đầu vào:** một ảnh RGB chứa một con chó hoặc mèo.
- **Đầu ra:** bản đồ nhãn cùng kích thước, mỗi điểm ảnh thuộc một trong 38 lớp: lớp 0 là nền, lớp 1–37 là giống của con vật (12 giống mèo, 25 giống chó).
- **Nhãn đúng:** ghép mặt nạ trimap chính thức (thú / nền / viền) với nhãn giống của ảnh. Điểm ảnh thuộc thú nhận mã giống, điểm nền nhận 0, vùng *viền* (dải mờ quanh biên con vật mà người gán nhãn không chắc chắn) nhận giá trị 255 và **bị bỏ qua** khi tính loss và metric.

1.2. Vì sao 38 lớp mà không chỉ tách thú khỏi nền

Đề bài yêu cầu tránh tập "quá dễ". Nếu chỉ tách thú khỏi nền (2–3 lớp), con vật thường to và nằm giữa ảnh nên các mô hình hiện đại đều đạt kết quả rất cao, khó thấy khác biệt giữa hai phương pháp. Bài toán 38 lớp khó hơn hẳn: mô hình phải khoanh đúng hình dạng *và* nhận ra đúng giống ở từng điểm ảnh, trong khi nhiều giống rất giống nhau (ví dụ American Pit Bull Terrier và Staffordshire Bull Terrier). Lỗi có thể nằm ở biên (sai hình dạng) hoặc ở nhãn (đúng hình dạng, sai giống), nên phân tích lỗi phong phú hơn. Đây cũng là cách nối tiếp A1: ở A1 nhóm thấy mô hình hay nhầm trong cùng một siêu lớp; ở A2 có thể hỏi tương tự là mô hình nhầm sang giống cùng loài hay nhầm chó thành mèo.

1.3. Câu hỏi nghiên cứu

1. **CNN hay Transformer?** Với cùng nguồn pretrained (ImageNet-1k) và số tham số tương đương, U-Net + ResNet-50 hay SegFormer-B2 cho mIoU cao hơn trên bài toán 38 lớp, và với chi phí (thời gian, bộ nhớ, tốc độ) ra sao?
2. **Lỗi đến từ đâu?** Phần mIoU bị mất là do vẽ sai biên hay do nhận sai giống? Mô hình nhầm giữa các giống cùng loài nhiều hơn nhầm chó ↔ mèo không?
3. **Mô hình nào cần ít nhãn hơn?** Khi chỉ còn 50%, 25%, 10% ảnh train, khoảng cách giữa hai mô hình thay đổi thế nào?
4. **Các tuyên bố của paper có đứng vững trên tập của nhóm không?** Ví dụ: bộ giải mã toàn MLP gọn nhẹ mà vẫn đủ tốt, encoder phân cấp không cần positional encoding (Mục 2).

2. Paper phân tích: SegFormer

2.1. Thông tin trích dẫn

E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, P. Luo. *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers*. Advances in Neural Information Processing Systems (**NeurIPS**), 2021. `https://arxiv.org/abs/2105.15203`. Mã nguồn chính thức: `https://github.com/NVlabs/SegFormer`.

2.2. Lý do chọn

- **Liên quan trực tiếp:** một trong hai phương pháp của nhóm chính là SegFormer, nên mọi thí nghiệm của nhóm đều đối chiếu được với paper (đúng yêu cầu "nối với phần cài đặt").
- **Hội nghị hàng đầu, ảnh hưởng lớn:** NeurIPS 2021; SegFormer là baseline Transformer phổ biến cho phân đoạn, có sẵn trọng số và cài đặt chính thức trong thư viện Hugging Face Transformers.
- **Vừa sức tính toán:** họ mô hình có nhiều cỡ (B0–B5); bản B2 chạy được trên GPU T4 của Colab.

2.3. Tóm tắt sơ bộ (sẽ phân tích đầy đủ ở A2.2)

Ý tưởng. Paper cho rằng một mô hình phân đoạn bằng Transformer không cần bộ giải mã phức tạp nếu encoder đã sinh được đặc trưng tốt ở nhiều tỉ lệ. Từ đó SegFormer gồm hai phần đơn giản:

- **Encoder Mix Transformer (MiT)** phân cấp, cho ra bốn bản đồ đặc trưng ở 1/4, 1/8, 1/16 và 1/32 độ phân giải ảnh (giống cấu trúc kim tự tháp của CNN, khác với ViT gốc chỉ có một tỉ lệ). Ba thay đổi chính so với ViT: ghép patch có chồng lấn (overlapping patch merging) để giữ liên tục cục bộ; self-attention "hiệu quả" rút gọn chiều dài chuỗi key/value theo một tỉ lệ R để giảm chi phí ở độ phân giải cao; và Mix-FFN chèn một tích chập depthwise 3×3 vào khối FFN thay cho positional encoding, nên mô hình không bị phụ thuộc vào kích thước ảnh lúc huấn luyện.
- **Bộ giải mã toàn MLP:** đưa bốn bản đồ đặc trưng về cùng số kênh bằng lớp MLP, phóng về 1/4 độ phân giải, ghép lại, trộn bằng một MLP nữa rồi dự đoán nhãn. Không có khối tích chập nặng hay module ngữ cảnh riêng.

Loss và huấn luyện. Cross-entropy theo từng điểm ảnh; AdamW, lịch learning rate "poly"; encoder pretrained trên ImageNet-1k. Nhóm dùng lại các lựa chọn này (Mục 6.2).

Thí nghiệm của paper. Đánh giá trên ADE20K (150 lớp), Cityscapes (19 lớp) và COCO-Stuff (171 lớp) bằng mIoU, kèm số tham số, FLOPs và FPS; so sánh với các mô hình CNN và Transformer cùng thời. Paper báo cáo SegFormer đạt mIoU cao hơn với ít tham số hơn các phương pháp trước, và bền vững hơn khi ảnh bị nhiễu (thử trên Cityscapes-C). Các con số cụ thể và việc đối chiếu sẽ được trình bày ở A2.2.

Điểm nhóm sẽ kiểm chứng. (1) Bộ giải mã toàn MLP chỉ dự đoán ở 1/4 độ phân giải: điều này có làm biên con vật (lông, tai, đuôi) kém sắc nét hơn U-Net, vốn phóng to dần tới độ phân giải gốc? (2) SegFormer-B2 có thắng một CNN mạnh cùng cỡ khi tập dữ liệu nhỏ hơn nhiều so với ADE20K/Cityscapes? (3) Chi phí thời gian và bộ nhớ thực tế trên GPU T4.

3. Tập dữ liệu: Oxford-IIIT Pet

3.1. Nguồn, license và cách tải

Oxford-IIIT Pet được nhóm Visual Geometry Group (Đại học Oxford) và IIIT Hyderabad công bố cùng bài báo CVPR 2012 [2], tại `https://www.robots.ox.ac.uk/~vgg/data/pets/`. Tác giả cung cấp dữ liệu cho mục đích nghiên cứu; việc dùng ảnh phải tôn trọng điều khoản của các trang web gốc nơi ảnh được thu thập (theo file `README` đi kèm bộ nhãn). Nhóm chỉ dùng cho học tập, không phân phối lại ảnh và trích dẫn bài báo gốc.

Mỗi ảnh có nhãn giống, nhãn loài (chó/mèo), hộp bao phần đầu và một mặt nạ **trimap** gán mỗi điểm ảnh là thú (1), nền (2) hoặc viền chưa rõ (3). Hai danh sách chính thức chia dữ liệu thành `trainval` (3.680 ảnh) và `test` (3.669 ảnh).

Máy chủ của Oxford không truy cập được từ mạng của nhóm, nên `data.py` thử máy chủ gốc trước rồi tải hai file `images.tar.gz` và `annotations.tar.gz` từ một bản sao trên Hugging Face. Hai file được **kiểm tra MD5** trùng với giá trị thư viện torchvision dùng cho bản gốc, nên dữ liệu giống hệt bản chính thức. Thư mục `data/` không được commit lên GitHub.

3.2. Cách chia split

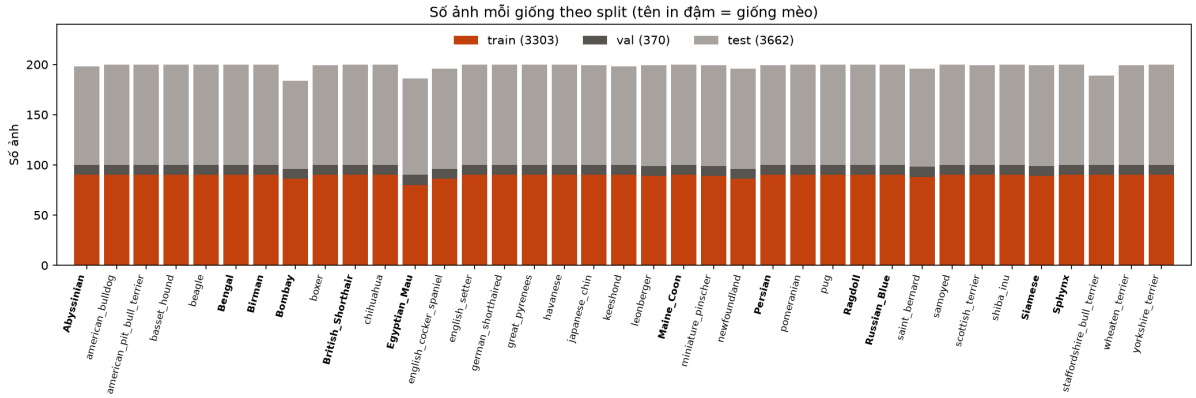
Bảng 1: Ba tập dữ liệu dùng chung cho mọi thí nghiệm (`splits/pet_split.json`, `output/eda/summary.json`).

Split	Số ảnh	Mỗi giống	Nguồn	Vai trò
Train	3.303	80–90	`trainval` chính thức	Cập nhật trọng số
Validation	370	10	`trainval` chính thức	Chọn checkpoint tốt nhất, theo dõi quá trình học
Test	3.662	88–100	`test` chính thức	Chỉ dùng một lần ở cuối mỗi thí nghiệm

Validation được lấy **phân tầng**: mỗi giống chọn ngẫu nhiên 10 ảnh từ danh sách `trainval` với seed 42 (`make_split` trong `data.py`). Tên ảnh của cả ba split được lưu cố định trong `splits/pet_split.json`. Tập test chính thức giữ nguyên và chỉ dùng một lần ở cuối mỗi thí nghiệm. Thí nghiệm ít dữ liệu ở A2.2 lấy tập con của train theo từng giống (`subsample`); tập nhỏ nằm trọn trong tập lớn hơn (10% $\subset$ 25% $\subset$ 50%), validation và test không đổi.

3.3. Khám phá dữ liệu (EDA)

Script `eda.py` quét toàn bộ 7.349 ảnh có trong hai danh sách chính thức (khoảng 20 giây trên Mac M1); kết quả lưu ở `output/eda/`.



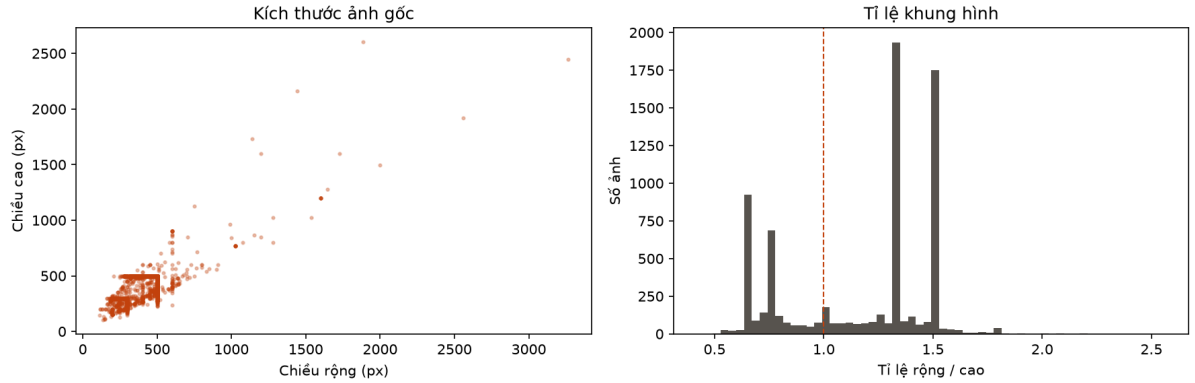
Hình 1: Số ảnh mỗi giống theo split. Tên in đậm là giống mèo.

Nhận xét về số lượng ảnh.

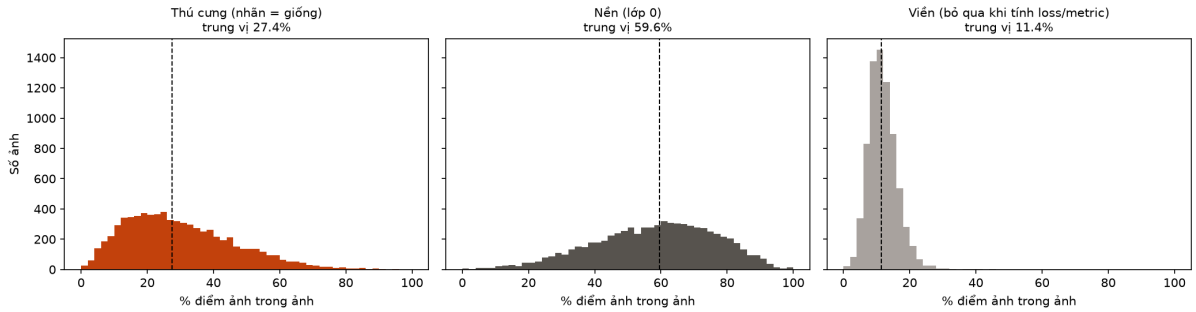
- Tập có 7.349 ảnh của **37 giống: 12 giống mèo và 25 giống chó**, mỗi giống khoảng 200 ảnh, chia gần đều giữa trainval và test (Hình 1). Vì số giống chó gấp đôi giống mèo, số ảnh chó cũng gấp đôi (train: 1.065 mèo, 2.238 chó).
- Dữ liệu **cân bằng theo giống**: mỗi giống có 80–90 ảnh train, 10 ảnh validation, 88–100 ảnh test. Ít nhất là Egyptian Mau và Bombay, do tập gốc có ít ảnh hơn và do bước loại ảnh lỗi bên dưới.
- **Phát hiện lỗi nhãn:** khi quét trimap, nhóm thấy **14 ảnh** (7 trong trainval, 7 trong test) có trimap gán *toàn bộ* ảnh là nền, dù ảnh rõ ràng có con vật (ví dụ `saint_bernard_108`, `Egyptian_Mau_162`, `Persian_259`). Giữ lại các ảnh này sẽ dạy mô hình rằng con vật là nền, và trừ điểm mô hình khi nó dự đoán đúng trên test. Nhóm **loại 14 ảnh này khỏi mọi split** và lưu danh sách trong `splits/pet_split.json` (`excluded_empty_trimap`). Vì vậy split có 3.303 / 370 / 3.662 ảnh thay vì 3.310 / 370 / 3.669.



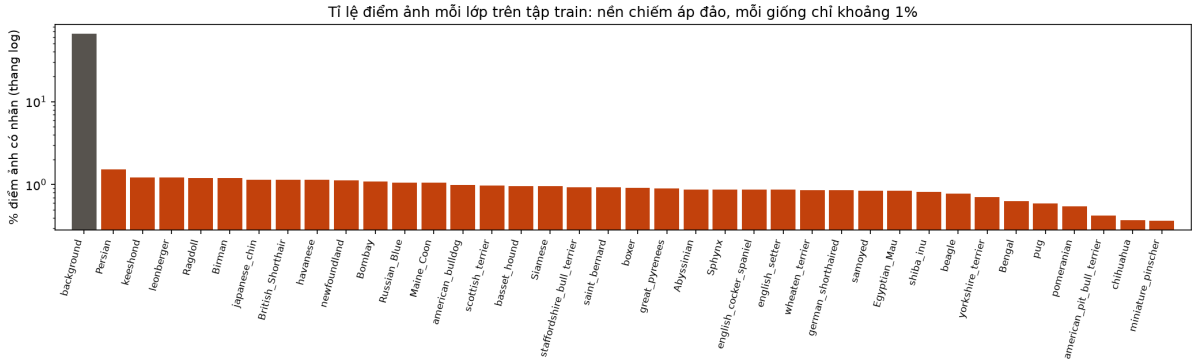
Hình 2: Ảnh mẫu (6 mèo, 6 chó trong tập train) và mặt nạ 38 lớp tương ứng.



Hình 3: Kích thước và tỉ lệ khung hình của ảnh gốc.



Hình 4: Phân bố tỉ lệ điểm ảnh thú / nền / viền trong mỗi ảnh (đường đứt nét: trung vị).



Hình 5: Tỉ lệ điểm ảnh có nhãn của mỗi lớp trên tập train (thang log), các giống xếp giảm dần.

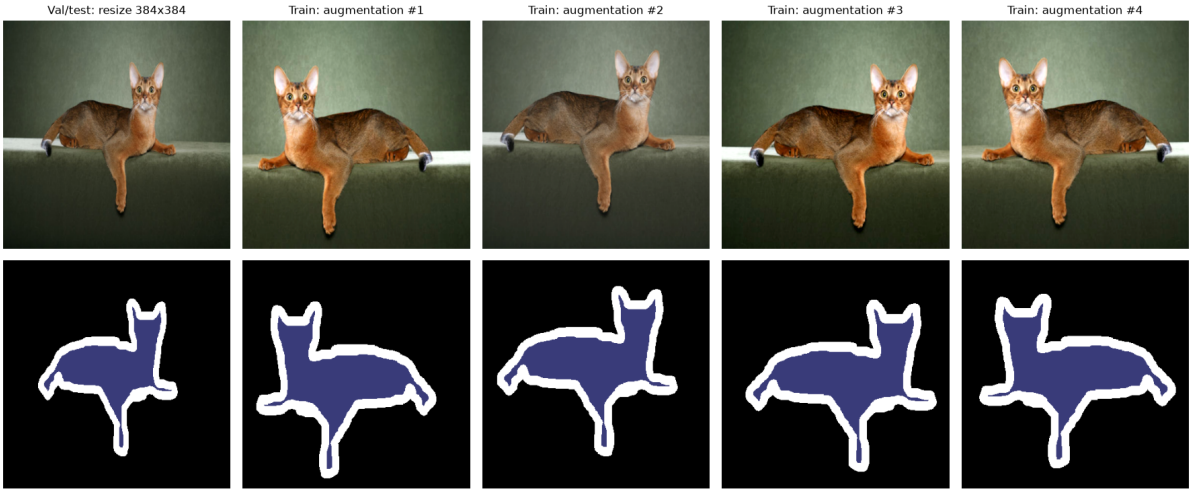
Nhận xét về ảnh và mặt nạ.

- **Kích thước ảnh không đồng nhất** (Hình 3): chiều rộng từ 114 đến 3.264 điểm ảnh, chiều cao từ 103 đến 2.606, trung vị 500×375; khoảng 98% ảnh có cạnh dài không quá 500. Tỉ lệ khung hình đa dạng, cả ngang lẫn dọc. Vì vậy cần đưa về một kích thước chung (384×384) để ghép batch. Mọi trimap đều cùng kích thước với ảnh.
- 7.346 ảnh là RGB, 3 ảnh có kênh alpha (RGBA); `data.py` luôn chuyển về RGB.
- **Con vật thường chiếm phần nhỏ của ảnh** (Hình 4): trung vị 27% điểm ảnh là thú, 60% là nền và 11% là viền. Dải viền khá dày (Hình 2) và bị bỏ qua, nên metric đo chất lượng ở phần thân con vật và nền rõ ràng, không đo độ sắc nét ở sát biên. Đây là giới hạn của nhãn gốc mà nhóm sẽ nêu khi thảo luận.
- **Mất cân bằng mạnh ở mức điểm ảnh** (Hình 5): trên tập train, nền chiếm 66% điểm ảnh có nhãn, mỗi giống chỉ 0,4–1,5%. Mô hình chỉ đoán "nền" ở mọi nơi đã đạt pixel accuracy 66% nhưng mIoU chỉ khoảng $66/38 \approx 1,7\%$. Đây là lý do nhóm chọn mIoU (trung bình đều các lớp) làm metric chính thay vì pixel accuracy.
- Trung bình pixel trên 500 ảnh train ngẫu nhiên (R, G, B) là (0,465; 0,430; 0,382), độ lệch chuẩn (0,265; 0,260; 0,266), gần thống kê ImageNet, nên dùng chuẩn hoá ImageNet là hợp lý.
- Trong một ảnh chỉ có một con vật và một giống, nên mô hình cần nhận ra giống từ toàn cảnh con vật rồi tô nhất quán cả vùng. Một lỗi điển hình có thể đoán trước là một con vật bị tô bằng hai giống khác nhau ở hai phần cơ thể.

4. DataLoader và augmentation

Bảng 2: Pipeline tiền xử lý, giống hệt nhau cho cả hai mô hình (`data.py`, torchvision `transforms.v2`).

Bước	Train	Validation / Test
Đọc dữ liệu	Ảnh JPEG chuyển sang RGB; trimap PNG chuyển thành mặt nạ 38 lớp (nền 0, giống 1–37, viền 255)	
Hình học (ảnh và mặt nạ cùng lúc)	`RandomResizedCrop(384, scale=(0.5, 1))` + `RandomHorizontalFlip`	`Resize` thẳng về 384×384
Màu (chỉ ảnh)	`ColorJitter(0.2, 0.2, 0.2)`	Không
Chuẩn hoá	Về $[0, 1]$, rồi `Normalize` theo mean/std của ImageNet	
DataLoader	batch 16, `shuffle`, `drop_last`	batch 16, không shuffle



Hình 6: Cùng một ảnh: bản dùng cho validation/test (trái) và bốn bản augmentation cho train, kèm mặt nạ đi theo.

Lý do thiết kế.

- **Biến đổi hình học áp dụng đồng thời cho ảnh và mặt nạ**, và mặt nạ luôn được nội suy theo điểm gần nhất (nearest) để không sinh ra mã lớp không tồn tại ở biên. `transforms.v2` của torchvision xử lý việc này khi mặt nạ được bọc trong kiểu `tv_tensors.Mask`; Hình 6 xác nhận mặt nạ khớp ảnh sau augmentation.
- **Kích thước** 384×384**:** ảnh gốc có cạnh khoảng 300–500 điểm ảnh (Mục EDA), nên 384 giữ được phần lớn chi tiết mà vẫn vừa bộ nhớ GPU T4 với batch 16. Cả hai mô hình nhận đúng cùng đầu vào; metric tính ở độ phân giải này.
- **Không lật dọc, không xoay mạnh:** ảnh thú cưng hầu như luôn đứng đúng chiều, nên các phép này chỉ tạo ảnh phi thực tế.
- **Chuẩn hoá ImageNet:** cả hai encoder được huấn luyện trước trên ImageNet-1k với cùng chuẩn hoá này.

5. Metric đánh giá

Mọi metric được tính trên **toàn bộ split** từ một ma trận nhầm lẫn 38×38 cộng dồn qua các ảnh (`metrics.py`), bỏ qua điểm ảnh viền. Gọi TP_c, FP_c, FN_c là số điểm ảnh đúng, dự đoán thừa và bỏ sót của lớp c:

$$\mathrm{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}, \quad \mathrm{mIoU} = \frac{1}{38}\sum_{c=0}^{37} \mathrm{IoU}_c, \quad \mathrm{Dice}_c = \frac{2TP_c}{2TP_c + FP_c + FN_c}.$$

- **mIoU 38 lớp (metric chính):** mỗi lớp có trọng số như nhau, nên giống hiếm hay khó cũng ảnh hưởng ngang lớp nền. Đây là metric chuẩn của bài toán phân đoạn ngữ nghĩa và cũng là metric chính của paper SegFormer.
- **Mean Dice:** trung bình Dice_c, metric thông dụng thứ hai mà đề bài nêu.
- **Pixel accuracy:** tỉ lệ điểm ảnh đúng. Chỉ báo cáo để tham khảo, vì nền chiếm phần lớn điểm ảnh (Hình 5) nên metric này dễ cao dù nhận sai giống.
- **IoU thú/nền:** gộp 37 giống thành một lớp "thú" rồi tính IoU. So với mIoU, nó tách riêng chất lượng khoanh hình dạng khỏi chất lượng nhận giống.
- **Chi phí:** số tham số, thời gian huấn luyện, bộ nhớ GPU đỉnh và tốc độ suy luận (ảnh/giây) trên cùng GPU T4.

6. Hai phương pháp và kế hoạch pipeline

6.1. Mô hình

Bảng 3: Hai mô hình so sánh; số tham số đo bằng `smoke_test.py` với đầu ra 38 lớp (`output/protocol_table.json`).

Họ	Mô hình	Tham số	Đặc điểm
CNN	U-Net + ResNet-50 `resnet50.a1_in1k` (timm)	32.521.606	Encoder 23.508.032 + bộ giải mã 9.013.574; giải mã dần tới độ phân giải gốc, có skip connection
Transformer	SegFormer-B2 `nvidia/mit-b2` (Hugging Face)	27.375.846	Encoder 24.196.288 + bộ giải mã MLP 3.179.558; dự đoán ở 1/4 độ phân giải

- **U-Net + ResNet-50** [3, 4]: encoder ResNet-50 (`timm` [6], trọng số `resnet50.a1_in1k`) cho 5 bản đồ đặc trưng ở 1/2 đến 1/32 độ phân giải. Bộ giải mã do nhóm tự viết (`models.py`): 5 bước, mỗi bước phóng gấp đôi, ghép với đặc trưng encoder cùng tỉ lệ (skip connection) và qua hai lớp tích chập 3×3 + BatchNorm + ReLU, cuối cùng là lớp 1×1 ra 38 kênh ở độ phân giải gốc.
- **SegFormer-B2** [1]: encoder MiT-B2 pretrained ImageNet-1k (`nvidia/mit-b2`) và bộ giải mã toàn MLP, lấy từ thư viện Hugging Face Transformers [7]. Đầu ra ở 1/4 độ phân giải được phóng bilinear về kích thước ảnh trước khi tính loss.

Hai mô hình có **cùng nguồn pretrained** (chỉ ImageNet-1k) và **số tham số cùng bậc**, để khác biệt kết quả đến từ kiến trúc chứ không phải từ dữ liệu pretrain hay kích thước. Nhóm cố ý không dùng các trọng số đã được huấn luyện cho phân đoạn trên COCO hay Pascal VOC, vì các tập này có sẵn lớp chó và mèo.

6.2. Huấn luyện

Bảng 4: Siêu tham số dự kiến cho A2.2, giống nhau cho cả hai mô hình (`config.TRAIN_HPARAMS`).

Thiết lập	Giá trị
Loss	Cross-entropy từng điểm ảnh, bỏ qua viền (`ignore_index=255`)
Optimizer	AdamW, weight decay 0,01
Learning rate	Encoder 6×10^{-5}; bộ giải mã và lớp ra 6×10^{-4} (gấp 10 lần, theo cấu hình huấn luyện của SegFormer)
Lịch learning rate	Warmup tuyến tính 1 epoch, sau đó "poly" (luỹ thừa 1) giảm về 0
Số epoch / batch / seed	40 / 16 / 42
Độ chính xác số	Mixed precision (fp16) trên GPU CUDA
Chọn checkpoint	Epoch có mIoU validation cao nhất; sau đó đánh giá trên test đúng một lần

Vòng lặp huấn luyện tự viết bằng PyTorch, dùng lại khung của A1 (`train.py` ở A1): checkpoint sau mỗi epoch để chạy tiếp khi Colab bị ngắt, ghi log JSON theo epoch, đo thời gian, bộ nhớ đỉnh và tốc độ suy luận. Thiết bị được chọn tự động theo thứ tự CUDA, MPS, CPU.

6.3. Ma trận thí nghiệm A2.2

Bảng 5: 8 lần huấn luyện dự kiến (`config.EXPERIMENTS`); dấu `*` là `unet_r50` hoặc `segformer_b2`.

Cấu hình	Mô hình	Ảnh train	Mục đích
`*_f100`	cả hai	3.303 (100%)	So sánh chính CNN vs Transformer
`*_f50`	cả hai	1.650 (50%)	Hiệu quả dữ liệu: mô hình nào giữ mIoU tốt hơn khi ít nhãn
`*_f25`	cả hai	812 (25%)	
`*_f10`	cả hai	332 (10%)	

6.4. Phân tích sẽ báo cáo ở A2.2

- Bảng mIoU, mean Dice, pixel accuracy, IoU thú/nền trên validation và test; đường loss/mIoU theo epoch.
- IoU từng giống; ma trận nhầm lẫn gộp theo nền / mèo / chó (`species_confusion` trong `metrics.py`) để tách lỗi "sai giống cùng loài" khỏi lỗi "sai loài".
- Ảnh lỗi điển hình của từng mô hình, so sánh chất lượng biên.
- Chi phí: thời gian, bộ nhớ, tốc độ suy luận; đường cong mIoU theo lượng dữ liệu train.
- **Mở rộng nếu kịp:** thử chéo trên ảnh chó/mèo của Pascal VOC 2012 [5] (chỉ đo IoU thú/nền vì VOC không có nhãn giống); demo dự đoán trên ảnh do người dùng tải lên.

7. Kiểm tra pipeline ở mốc A2.1

Script `smoke_test.py` lấy một batch thật từ DataLoader train (2 ảnh 384×384, mặt nạ chứa nền, hai mã giống và giá trị viền 255), dựng cả hai mô hình với trọng số pretrained,

rồi kiểm tra: (1) đầu ra có đúng kích thước $2 \times 38 \times 384 \times 384$; (2) sau một lần backward, cả encoder lẫn bộ giải mã đều nhận gradient; (3) khi học thử 10 bước trên chính batch đó, loss giảm. Kết quả chạy trên Mac M1 (thiết bị `mps`), lưu ở `output/protocol_table.json`:

Bảng 6: Kết quả kiểm tra pipeline (`smoke_test.py`).

Mô hình	Kích thước đầu ra	Gradient encoder + decoder	Loss bước 1 → bước 10
U-Net + ResNet-50	$2 \times 38 \times 384 \times 384$	Có	3,811 → 3,080
SegFormer-B2	$2 \times 38 \times 384 \times 384$	Có	3,686 → 0,048

Loss ban đầu xấp xỉ $\ln 38 \approx 3{,}64$, đúng mức của một bộ phân loại 38 lớp chưa học, và giảm ở cả hai mô hình: dữ liệu, nhãn, mô hình, loss (có bỏ qua viền) và optimizer với hai learning rate đã nối với nhau đúng. Tốc độ giảm loss trên một batch 2 ảnh **không phải kết quả so sánh**: đây chỉ là kiểm tra code. Thời gian mỗi bước trên máy Mac 8 GB cũng không dùng để so sánh vì máy bị thiếu bộ nhớ khi chạy; chi phí thật sẽ được đo trên GPU T4 ở A2.2. Ngoài ra, `metrics.py` được kiểm tra trên các mặt nạ nhỏ tính tay: dự đoán đúng hoàn toàn cho mIoU, pixel accuracy và IoU thú/nền đều bằng 1; dự đoán "toàn nền" cho IoU thú/nền bằng 0 và điểm ảnh viền không được tính.

DataLoader với batch 16 cho 206 batch train, 24 batch validation và 229 batch test mỗi epoch.

8. Kế hoạch đến mốc A2.2

Bảng 7: Lịch làm việc đến hạn A2.2 (16/11/2026).

Thời gian	Công việc
03/11 – 05/11	Viết `train.py` (dựa trên A1), notebook Colab; chạy thử trên tập con trên máy Mac M1
05/11 – 09/11	Huấn luyện 8 cấu hình trên Google Colab (GPU T4), lưu log và checkpoint
09/11 – 11/11	Viết `analyze.py`: bảng, biểu đồ, IoU từng giống, ma trận nhầm lẫn, ảnh lỗi; thử chéo trên Pascal VOC
10/11 – 13/11	Phân tích đầy đủ paper SegFormer và đối chiếu với kết quả của nhóm
12/11 – 16/11	Hoàn thiện báo cáo A2, landing page (đủ 5 mục), demo và video

Rủi ro và cách xử lý.

- **Hết hạn mức GPU Colab:** checkpoint sau mỗi epoch; nếu vẫn thiếu, giảm số epoch của các cấu hình ít dữ liệu trước, giữ nguyên hai cấu hình chính, và ghi rõ trong báo cáo.
- **Bộ nhớ GPU:** nếu batch 16 ở 384×384 vượt bộ nhớ T4, giảm batch và tích luỹ gradient để giữ batch hiệu dụng 16.
- **Giống ít ảnh hoặc rất giống nhau** làm mIoU dao động: báo cáo thêm IoU từng giống và IoU thú/nền để đọc đúng nguyên nhân.

9. Sử dụng công cụ AI

Theo quy định liêm chính học thuật của môn học, nhóm ghi rõ: trợ lý AI (Claude) được dùng để hỗ trợ viết khung mã nguồn, soạn bố cục báo cáo và rà soát lỗi. Nhóm đã đọc, chạy kiểm tra, chỉnh sửa và chịu trách nhiệm về toàn bộ nội dung; mọi lựa chọn chủ đề, paper, dataset và mô hình đều được nhóm hiểu và có thể giải thích.

Tài liệu

[1] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, P. Luo. SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. *NeurIPS*, 2021. `https://arxiv.org/abs/2105.15203`.

[2] O. M. Parkhi, A. Vedaldi, A. Zisserman, C. V. Jawahar. Cats and Dogs. *CVPR*, 2012.

[3] O. Ronneberger, P. Fischer, T. Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI*, 2015.

[4] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.

[5] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman. The Pascal Visual Object Classes (VOC) Challenge. *IJCV*, 2010.

[6] R. Wightman. PyTorch Image Models (timm). `https://github.com/huggingface/pytorch-image-models`.

[7] T. Wolf et al. Transformers: State-of-the-Art Natural Language Processing. *EMNLP (System Demonstrations)*, 2020.